

Pandas DataFrames & Merging — Class Practice 7.3

Jupyter Notebook · `pd.read_csv` · DataFrame Merging · inner / left / right / outer · Missing Data

1. Scenario

GlobalTrade Ltd. — Order & Shipment Analysis System

You are a data analyst at GlobalTrade Ltd., an international e-commerce company. The operations team stores customer orders and shipment records in two separate CSV files. Your manager needs you to build a Jupyter Notebook that reads both CSV files into Pandas DataFrames, cleans and processes each one, and merges them to produce a complete order-fulfilment report — a workflow used every day in real data analytics roles.

Your completed notebook must:

1. Load both CSV files into Pandas DataFrames using `pd.read_csv()`.
2. Inspect and clean each DataFrame — check shapes, dtypes, and missing values.
3. Perform all four types of DataFrame merge: inner, left, right, and outer.
4. Add calculated columns to the merged result and flag unmatched rows.
5. Summarise the merged data using groupby and aggregation.
6. Document every step with a Markdown cell above each Code cell.

CSV File Setup — do this BEFORE opening Jupyter

Two CSV files are provided with this exercise. Save both into the same folder as your notebook:

`orders.csv` — 10 customer order records (5 columns)

`shipments.csv` — 8 shipment records (4 columns)

Your notebook file `DA_Exercise_7_3.ipynb` and BOTH CSV files must be in the same folder.

Verify by running: `import os; print(os.listdir('.'))` in a Code cell before you start.

Jupyter Setup

Launch Jupyter: `jupyter notebook` in your terminal.
Navigate to the folder containing `orders.csv` and `shipments.csv`.

Create a new notebook — File > New > Notebook > Python 3 (ipykernel).
Rename it: DA_Exercise_7_3.ipynb
Run each Code cell with Shift+Enter. Add Markdown cells with Esc → M.

2. CSV File Contents

Study both CSV files before writing any code. Notice which order_ids appear in each file — this determines what each merge type will produce.

orders.csv (10 data rows + header)				
order_id	customer	product	quantity	unit_price
101	Chen	Laptop	2	899.99
102	Patel	Tablet	5	499.50
103	Müller	Monitor	1	349.00
104	Okonkwo	Keyboard	10	79.99
105	Santos	Headset	3	149.99
106	Larsson	Webcam	4	89.99
107	Kim	Mouse	8	29.99
108	O'Brien	Docking Station	1	199.00
109	Rossi	SSD	6	119.99
110	Nakamura	USB Hub	2	39.99

shipments.csv (8 data rows + header)			
order_id	ship_date	carrier	status
101	2024-01-10	DHL	Delivered
102	2024-01-12	FedEx	Delivered
103	2024-01-13	DHL	In Transit
104	2024-01-14	UPS	Delivered
105	2024-01-15	FedEx	Delayed
106	2024-01-16	UPS	Delivered
107	2024-01-18	DHL	In Transit
111	2024-01-19	FedEx	Delivered

Key Observations

Shared key: `order_id` — this column appears in BOTH files and is used for all merges.

order_ids 101–107 appear in BOTH files → these produce matched rows in every merge.

order_ids 108, 109, 110 appear in `orders.csv` only → left-only rows (unshipped orders).

order_id 111 appears in `shipments.csv` only → right-only row (orphan shipment).

The four merge types — know these before you start

how=	Syntax	Rows kept	Keys	When to use
inner	<code>pd.merge(df_o, df_s, on='order_id', how='inner')</code>	Rows where key exists in BOTH	Intersection	Only fully matched records
left	<code>pd.merge(df_o, df_s, on='order_id', how='left')</code>	All LEFT rows; NaN where no right match	All left keys	Find unshipped orders
right	<code>pd.merge(df_o, df_s, on='order_id', how='right')</code>	All RIGHT rows; NaN where no left match	All right keys	Find orphan shipments
outer	<code>pd.merge(df_o, df_s, on='order_id', how='outer')</code>	ALL rows from both; NaN where no match	Union	Full audit — nothing lost

3. Requirements

Your notebook must satisfy all six requirements below. Each requirement maps to one or more Code cells, each preceded by a Markdown cell.

1

Req

Notebook Header — Markdown Cell

Add a Markdown cell at the very top containing:

```
# Data Analytics — Exercise 7.3: Pandas DataFrames & Merging
```

```
## Your Name | Date
```

A brief (2-sentence) description of the business problem this notebook solves.

2

Req

Load Both CSV Files with `pd.read_csv()`

import pandas as pd — first Code cell.

Load the CSV files using `pd.read_csv()`:

```
df_orders = pd.read_csv('orders.csv')
```

```
df_shipments = pd.read_csv('shipments.csv')
```

For each DataFrame print: the first 5 rows (`.head()`), shape, dtypes, and `isnull().sum()`.

Verify both load without errors before continuing.

3

Req

Inspect and Clean Both DataFrames

On df_orders: add a calculated column 'order_value' = quantity * unit_price.
 On df_orders: use df.describe() to print numeric summary statistics.
 On df_shipments: convert 'ship_date' from string to datetime using pd.to_datetime().
 Verify the conversion by printing df_shipments.dtypes again.
 Print the total order value across all orders using an f-string formatted to 2dp.

4

Req

Perform All Four Types of DataFrame Merge

inner merge: pd.merge(df_orders, df_shipments, on='order_id', how='inner')
 → Print the result. Note which order_ids appear and which were dropped.
 left merge: pd.merge(df_orders, df_shipments, on='order_id', how='left')
 → Print the result. Identify unshipped orders (rows where 'carrier' is NaN).
 right merge: pd.merge(df_orders, df_shipments, on='order_id', how='right')
 → Print the result. Identify the orphan shipment (order_id with no order record).
 outer merge: pd.merge(df_orders, df_shipments, on='order_id', how='outer')
 → Print shape and total NaN count using isnull().sum().sum().

5

Req

Add Calculated Columns and Flag Unmatched Rows

Using the LEFT merged DataFrame:
 df_left['shipped'] = df_left['carrier'].notna() # True / False flag
 Print the updated DataFrame showing order_id, customer, order_value, shipped.
 Count shipped vs unshipped with value_counts().
 Calculate and print the fulfilment rate to 2dp: shipped / total * 100

6

Req

GroupBy and Summary on the Inner Merged Result

Using the INNER merged DataFrame (only fully matched orders):
 groupby('status')['order_value'].agg(['sum','mean','count'])
 groupby('carrier')['order_value'].sum().sort_values(ascending=False)
 Print the carrier totals and identify the top carrier using .index[0].
 groupby('status')['quantity'].sum() — units dispatched per status
 Write a Markdown Summary cell with 3 sentences interpreting the fulfilment picture.

4. Hints

Pandas merge quick reference — compare the given example vs your task

Given example (students & grades)	Your task (orders & shipments from CSV)
<pre>import pandas as pd df_s = pd.read_csv('students.csv')</pre>	<pre>import pandas as pd df_orders = pd.read_csv('orders.csv')</pre>

<code>df_g = pd.read_csv('grades.csv')</code>	<code>df_shipments = pd.read_csv('shipments.csv')</code>
<code>df_s.head()</code> <code>df_g.shape</code>	<code>df_orders.head()</code> <code>df_shipments.shape</code>
<code>df_s['total'] = df_s['score'] * 1.0</code>	<code>df_orders['order_value'] = df_orders['quantity'] * df_orders['unit_price']</code>
<code>pd.merge(df_s, df_g, on='id', how='inner')</code>	<code>pd.merge(df_orders, df_shipments, on='order_id', how='inner')</code>
<code>pd.merge(df_s, df_g, on='id', how='left')</code>	<code>pd.merge(df_orders, df_shipments, on='order_id', how='left')</code>
<code>df_left['grade'].isna()</code> # ungraded students	<code>df_left['carrier'].isna()</code> # unshipped orders

1

Hint

Loading CSV files — file path and read_csv options

```
df_orders = pd.read_csv('orders.csv')
Both CSV files must be in the SAME folder as your .ipynb notebook.
If you get FileNotFoundError, check the folder with: import os;
os.listdir('.')
To load from a subfolder: pd.read_csv('data/orders.csv')
Add parse_dates=['ship_date'] to convert dates automatically:
df_shipments = pd.read_csv('shipments.csv',
parse_dates=['ship_date'])
```

2

Hint

Inspecting after loading — what to check first

```
print(df_orders.shape)                    # should be (10, 5)
print(df_shipments.shape)                # should be (8, 4)
print(df_orders.dtypes)                  # unit_price should be float64
print(df_orders.isnull().sum())        # check for unexpected NaN
values
If a column has the wrong dtype, use .astype() to fix it before merging.
```

3

Hint

Converting ship_date and working with dates

```
df_shipments['ship_date'] =
pd.to_datetime(df_shipments['ship_date'])
OR load with parse_dates in read_csv (Hint 1 shows both
approaches).
After conversion, dtype changes from 'object' to 'datetime64[ns]'.
Extract the month: df_shipments['ship_date'].dt.month
```

4

Hint

Detecting NaN rows after each merge type

```
After LEFT merge — find unshipped orders (NaN in carrier):
unshipped = df_left[df_left['carrier'].isna()]
print(f'Unshipped: {list(unshipped["order_id"])}')
After RIGHT merge — find orphan shipment (NaN in customer):
```

```
orphan = df_right[df_right['customer'].isna()]
print(f'Orphan order_id: {orphan["order_id"].values[0]}')
```

5 Fulfilment rate and GroupBy on inner merge

Hint

```
df_left['shipped'] = df_left['carrier'].notna() # True/False column
rate = df_left['shipped'].sum() / len(df_left) * 100
print(f'Fulfilment rate: {rate:.2f}%')

GroupBy on inner merge:
inner.groupby('status')['order_value'].agg(['sum', 'mean', 'count'])
carrier =
inner.groupby('carrier')['order_value'].sum().sort_values(ascending=False)
print(f'Top carrier: {carrier.index[0]} - £{carrier.iloc[0]:.2f}')
```

5. Solution

Attempt all six requirements in your notebook before reading the solution. Use the hints, the CSV file previews, and the merge-types table as your guide.

DA_Exercise_7_3.ipynb — complete notebook cell sequence

CELL 1 — Markdown cell (Esc → M before typing)

➔ **Markdown** — Notebook header

```
# Data Analytics — Exercise 7.3: Pandas DataFrames & Merging
## Your Name | Date: 2025-02-01
```

This notebook reads GlobalTrade Ltd.'s order and shipment CSV files into Pandas DataFrames, cleans both datasets, and merges them using all four join types to produce a complete order-fulfilment report.

CELL 2 — Markdown cell

➔ **Markdown** — Section heading — load CSV files

```
## 1. Load CSV Files with pd.read_csv()
We load orders.csv and shipments.csv from the current folder.
Both files must be in the same directory as this notebook.
```

CELL 3 — Code cell (Shift+Enter to run)

```
In [ ]: Requirement 2 — load both CSV files
# Requirement 1 — import Pandas
import pandas as pd
```

```
# Requirement 2 — load CSV files with pd.read_csv()
df_orders = pd.read_csv('orders.csv')
df_shipments = pd.read_csv('shipments.csv')

print('— orders.csv loaded —————')
print(df_orders.head())
print(f'shape : {df_orders.shape}')
print(f'dtypes:\n{df_orders.dtypes}')
print(f'missing: {df_orders.isnull().sum().sum()}')

print('\n— shipments.csv loaded —————')
print(df_shipments.head())
print(f'shape : {df_shipments.shape}')
print(f'dtypes:\n{df_shipments.dtypes}')
print(f'missing: {df_shipments.isnull().sum().sum()}')
```

Out []: output

```
— orders.csv loaded —————
  order_id customer  product  quantity  unit_price
0        101     Chen   Laptop         2      899.99
1        102    Patel   Tablet         5      499.50
2        103  Müller  Monitor         1      349.00
3        104 Okonkwo  Keyboard        10       79.99
4        105   Santos  Headset         3      149.99
shape : (10, 5)
dtypes: order_id int64  customer object  unit_price float64
missing: 0

— shipments.csv loaded —————
  order_id  ship_date carrier  status
0        101  2024-01-10    DHL  Delivered
1        102  2024-01-12  FedEx  Delivered
2        103  2024-01-13    DHL  In Transit
3        104  2024-01-14    UPS  Delivered
4        105  2024-01-15  FedEx  Delayed
shape : (8, 4)
dtypes: order_id int64  ship_date object  carrier object  status object
missing: 0
```

CELL 4 — Markdown cell

— **Markdown** — Section heading — inspect and clean

2. Inspect and Clean Both DataFrames

We calculate order_value, run describe(), and convert ship_date to datetime.

CELL 5 — Code cell

In []: Requirement 3 — clean and inspect

```
# Add order_value = quantity × unit_price (element-wise, no loop)
df_orders['order_value'] = df_orders['quantity'] * df_orders['unit_price']
```

```

print('— Orders with order_value —————')
print(df_orders[['order_id', 'customer', 'product', 'order_value']])

print('\n— Numeric summary —————')
print(df_orders.describe())

# Convert ship_date string → datetime64
df_shipments['ship_date'] = pd.to_datetime(df_shipments['ship_date'])
print('\n— df_shipments dtypes after conversion ———')
print(df_shipments.dtypes)

total = df_orders['order_value'].sum()
print(f'\nTotal order value: £{total:,.2f}')

```

Out[:]: output

```

— Orders with order_value —————
   order_id  customer      product  order_value
0        101      Chen        Laptop      1799.98
1        102      Patel        Tablet      2497.50
2        103    Müller        Monitor      349.00
...      (10 rows total)
9        110  Nakamura      USB Hub       79.98

— df_shipments dtypes after conversion ———
order_id      int64
ship_date    datetime64[ns]
carrier      object
status       object

Total order value: £7,495.15

```

CELL 6 — Markdown cell

— **Markdown** — Section heading — four merge types

3. Performing All Four DataFrame Merge Types

We merge `df_orders` and `df_shipments` on 'order_id' using inner, left, right, and outer.

Refer to the merge-types table in the exercise sheet to predict each result.

CELL 7 — Code cell

In [:]: Requirement 4 — all four merge types

```

# — INNER — only rows where order_id exists in BOTH files —————
df_inner = pd.merge(df_orders, df_shipments, on='order_id', how='inner')
print('— INNER merge —————')
print(df_inner[['order_id', 'customer', 'order_value', 'carrier', 'status']])
print(f'shape: {df_inner.shape} (7 matched rows — 108/109/110/111 excluded)')

# — LEFT — all 10 orders; shipment cols NaN for 108, 109, 110 —————
df_left = pd.merge(df_orders, df_shipments, on='order_id', how='left')
print('\n— LEFT merge —————')

```

```

print(df_left[['order_id', 'customer', 'order_value', 'carrier', 'status']])
unshipped = df_left[df_left['carrier'].isna()]
print(f'Unshipped orders ({len(unshipped)}): {list(unshipped["order_id"])}')

# — RIGHT — all 8 shipments; order cols NaN for orphan (111) —————
df_right = pd.merge(df_orders, df_shipments, on='order_id', how='right')
print('\n— RIGHT merge —————')
print(df_right[['order_id', 'customer', 'carrier', 'status']])
orphan = df_right[df_right['customer'].isna()]
print(f'Orphan shipment: order_id {orphan["order_id"].values[0]}')

# — OUTER — ALL rows from both; NaN wherever no match —————
df_outer = pd.merge(df_orders, df_shipments, on='order_id', how='outer')
print('\n— OUTER merge —————')
print(df_outer[['order_id', 'customer', 'carrier', 'status']])
print(f'shape: {df_outer.shape} | total NaN values: {df_outer.isnull().sum().sum()}')

```

Out[:]: output

```

— INNER merge —————
   order_id customer  order_value carrier  status
0         101     Chen      1799.98    DHL  Delivered
1         102     Patel      2497.50  FedEx  Delivered
2         103   Müller       349.00    DHL  In Transit
3         104 Okonkwo       799.90    UPS  Delivered
4         105   Santos       449.97  FedEx   Delayed
5         106 Larsson       359.96    UPS  Delivered
6         107      Kim       239.92    DHL  In Transit
shape: (7, 9)  (7 matched rows — 108/109/110/111 excluded)

```

```

— LEFT merge —————
   order_id customer  order_value carrier  status
0         101     Chen      1799.98    DHL  Delivered
...  (rows 1-6 shown)...
7         108  O'Brien       199.00    NaN     NaN
8         109     Rossi       719.94    NaN     NaN
9         110 Nakamura        79.98    NaN     NaN
Unshipped orders (3): [108, 109, 110]

```

```

— RIGHT merge —————
   order_id customer carrier  status
0         101     Chen    DHL  Delivered
...  (rows 1-6)...
7         111      NaN  FedEx  Delivered ← orphan
Orphan shipment: order_id 111

```

```

— OUTER merge —————
shape: (11, 9) | total NaN values: 16

```

CELL 8 — Markdown cell

➡ **Markdown** — Section heading — shipped flag and fulfilment

4. Shipped Flag and Fulfilment Rate

Using the LEFT merged DataFrame we flag each order as shipped or unshipped and calculate the overall fulfilment rate for the current period.

CELL 9 — Code cell

In []: Requirement 5 — shipped flag and fulfilment rate

```
# notna() returns True where the value is NOT NaN
df_left['shipped'] = df_left['carrier'].notna()

shipped_count = df_left['shipped'].sum()
total_orders = len(df_left)
fulfilment_rate = shipped_count / total_orders * 100

print('— Shipped Flag per Order —')
print(df_left[['order_id', 'customer', 'order_value', 'shipped']])

print('\n— Shipped vs Unshipped Count —')
print(df_left['shipped'].value_counts())
print(f'\nFulfilment rate: {fulfilment_rate:.2f}%\n({shipped_count}/{total_orders} orders)')
```

Out []: output

```
— Shipped Flag per Order —
  order_id  customer  order_value  shipped
0       101      Chen      1799.98    True
1       102      Patel      2497.50    True
...  (rows 2-6) ...
7       108  O'Brien       199.00   False
8       109      Rossi       719.94   False
9       110  Nakamura        79.98   False
```

```
— Shipped vs Unshipped Count —
shipped
True      7
False     3
Name: shipped, dtype: int64
```

Fulfilment rate: 70.00% (7/10 orders)

CELL 10 — Markdown cell

➔ **Markdown** — Section heading — GroupBy summary

5. GroupBy Summary on the Inner Merged DataFrame

We analyse performance by delivery status and by carrier.

The inner merge is used here — it contains only fully matched orders.

CELL 11 — Code cell

In []: Requirement 6 — GroupBy on inner merge

```
# Summary by delivery status
status_summary =
df_inner.groupby('status')['order_value'].agg(['sum', 'mean', 'count'])
print('— Order Value by Delivery Status —')
print(status_summary.round(2))

# Carrier totals — sorted highest first
carrier_totals =
df_inner.groupby('carrier')['order_value'].sum().sort_values(ascending=False)
print('\n— Carrier Totals (descending) —')
print(carrier_totals)
print(f'Top carrier: {carrier_totals.index[0]} —
£{carrier_totals.iloc[0]:,.2f}')

# Units dispatched per delivery status
units_per_status = df_inner.groupby('status')['quantity'].sum()
print('\n— Units Dispatched by Status —')
print(units_per_status)
```

Out[:]: output

```
— Order Value by Delivery Status —
      sum      mean  count
status
Delayed    449.97   449.97     1
Delivered  5456.34  1364.09     4
In Transit   589.06   294.53     2

— Carrier Totals (descending) —
carrier
FedEx    2947.47
DHL      2388.90
UPS       1159.86
Top carrier: FedEx — £2,947.47

— Units Dispatched by Status —
status
Delayed      3
Delivered    22
In Transit    9
```

CELL 12 — Markdown cell

— Markdown — Summary interpretation

```
## Summary

- The fulfilment rate is 70% (7 of 10 orders shipped);
  orders 108, 109, and 110 are still awaiting dispatch.
- FedEx is the highest-value carrier at £2,947 across 2 shipments;
  only 1 order (Santos, Headset) is currently delayed.
- The orphan shipment (order_id 111) has no matching order record —
  this data quality issue should be escalated to the warehouse team.
```

6. Expected Output

After loading the provided CSV files and running Kernel > Restart & Run All, your notebook should produce the following:

Notebook Output

```
— Cell 3 — CSV files loaded —————
df_orders      shape : (10, 5) | missing: 0
df_shipments shape : (8, 4)  | missing: 0
df_orders dtypes : int64, object, object, int64, float64
df_shipments dtypes: int64, object (ship_date — convert next)

— Cell 5 — inspect and clean —————
Total order value: £7,495.15
ship_date dtype after pd.to_datetime() → datetime64[ns]

— Cell 7 — all four merge types —————
INNER shape: (7, 9)  — order_ids 101-107 only
LEFT  shape: (10, 9) — all 10 orders; 108/109/110 have NaN shipment cols
RIGHT shape: (8, 9)  — all 8 shipments; order_id 111 has NaN order cols
OUTER shape: (11, 9) — 11 rows total; 16 NaN values
Unshipped orders (3): [108, 109, 110]
Orphan shipment   (1): order_id 111

— Cell 9 — shipped flag —————
shipped: True=7   False=3
Fulfilment rate: 70.00% (7/10 orders)

— Cell 11 — GroupBy summary —————
Delivered — sum £5,456.34 mean £1,364.09 count 4
In Transit — sum  £589.06 mean  £294.53 count 2
Delayed    — sum  £449.97 mean  £449.97 count 1
Top carrier: FedEx — £2,947.47
Units: Delivered 22 | In Transit 9 | Delayed 3
```

Cell breakdown — what each cell produces

Cell	Type	Requirement	Key Output
1	Markdown	Header	Formatted notebook title and business description
2	Markdown	Intro	Section heading for CSV loading
3	Code	Req 2	Both DataFrames loaded from CSV; head(), shape, dtypes, missing

4	Markdown	Intro	Section heading for inspect and clean
5	Code	Req 3	order_value column; describe(); datetime conversion; total £7,495.15
6	Markdown	Intro	Section heading for four merge types
7	Code	Req 4	All 4 merges; shapes; unshipped [108,109,110]; orphan 111
8	Markdown	Intro	Section heading for shipped flag
9	Code	Req 5	shipped column; value_counts; fulfilment rate 70.00%
10	Markdown	Intro	Section heading for GroupBy
11	Code	Req 6	Status & carrier GroupBy; top carrier FedEx; units per status
12	Markdown	Summary	3-sentence written interpretation of the fulfilment picture

7. Pandas Merge Quick Reference

Every Pandas concept used in this exercise, mapped to its syntax and purpose.

Concept	Syntax used	What it does
Read CSV	<code>pd.read_csv('file.csv')</code>	Loads a CSV file into a DataFrame
Read CSV + parse dates	<code>pd.read_csv('f.csv', parse_dates=['col'])</code>	Converts named column to datetime on load
DataFrame shape	<code>df.shape</code>	Returns (rows, columns) tuple
Column types	<code>df.dtypes</code>	Shows dtype of every column
Missing count	<code>df.isnull().sum()</code>	NaN count per column
Total missing	<code>df.isnull().sum().sum()</code>	Single count of all NaN cells

Add column	<code>df['c'] = df['a'] * df['b']</code>	Element-wise calculation stored as new column
Describe numerics	<code>df.describe()</code>	Count, mean, std, min, quartiles, max
Convert to datetime	<code>pd.to_datetime(df['col'])</code>	Parses date strings to <code>datetime64[ns]</code>
Inner merge	<code>pd.merge(a, b, on='key', how='inner')</code>	Only rows where key exists in both
Left merge	<code>pd.merge(a, b, on='key', how='left')</code>	All left rows; NaN where no right match
Right merge	<code>pd.merge(a, b, on='key', how='right')</code>	All right rows; NaN where no left match
Outer merge	<code>pd.merge(a, b, on='key', how='outer')</code>	All rows from both; NaN wherever no match
Find NaN rows	<code>df[df['col'].isna()]</code>	Filter to rows where column is NaN
<code>notna()</code> flag	<code>df['col'].notna()</code>	True where value is NOT NaN
<code>value_counts()</code>	<code>df['col'].value_counts()</code>	Frequency of each unique value
GroupBy multi-stat	<code>df.groupby('col').agg({'val': ['sum', 'mean', 'count']})</code>	Multiple stats per group
GroupBy sort	<code>df.groupby('col')['val'].sum().sort_values(ascending=False)</code>	Sorted group totals
Top group	<code>series.index[0]</code>	Label of the first (highest) group after sort

8. Extension Challenges

Once your base notebook runs correctly with Kernel > Restart & Run All, try these extensions:

- Add a Code cell that re-loads `df_shipments` using `pd.read_csv('shipments.csv', parse_dates=['ship_date'])` instead of calling `pd.to_datetime()` manually. Confirm the dtype is `datetime64[ns]` without the conversion step and explain the difference in a Markdown cell.
- Add a Code cell that uses `pd.merge(df_orders, df_shipments, on='order_id', how='left', indicator=True)`. The `_merge` column shows 'both', 'left_only', or 'right_only' for each row. Print `value_counts()` of this column and explain what each label reveals about the data.
- Add a Code cell that extracts the ship month from `df_shipments['ship_date'].dt.month_name()` and groups the inner merge by month to compute total order value per month. Print the result and identify which month had the highest dispatch value.
- Simulate a data quality fix: in a new Code cell, manually add a row to `df_orders` for `order_id` 111 (create a realistic customer name and product). Re-run the inner merge and verify that `order_id` 111 now appears in the result instead of being an orphan.
- Add a Code cell that exports the LEFT merged DataFrame to a new CSV: `df_left.to_csv('fulfilment_report.csv', index=False)`. Re-read it with `pd.read_csv()` and verify the shape and columns match the original left merge DataFrame.
- Add a Code cell that uses `df_inner.pivot_table(values='order_value', index='carrier', columns='status', aggfunc='sum', fill_value=0)` to create a pivot showing order value split by carrier and status. Print the pivot table and discuss what it reveals.